Simulación: el robot guardián del modelo (Sesión 5)

El caso. El banco prepara la **v1.1** del modelo de riesgo de crédito. Después del incidente del AUC 0.27, el comité de Riesgos aprobó tres reglas nuevas:

1. **Ningún modelo con AUC de test menor a 0.70 puede integrarse ni llegar a producción.**
2. **Ninguna contraseña puede estar escrita en el código.**
3. **Todo despliegue a producción debe ser aprobado por el líder técnico.**

Su misión: convertir el robot de CI de la sesión pasada en un **robot guardián** que haga cumplir estas reglas solo.

Roles (las mismas parejas de la Sesión 3, en su repo `sprint-equipo-<N>`)

Rol	Responsabilidades
MLOps Lead (antes DS Senior, admin del repo)	Configura la bóveda (secrets, variables, environment) y las reglas de protección. Revisa y aprueba PRs. Aprueba el despliegue a producción.
Data Scientist (antes DS Junior)	Escribe el workflow del robot guardián. En la Misión 4 intenta "colar" un cambio que empeora el modelo. Prepara la release.

*Si un equipo tiene 3 integrantes, el tercero es **Auditor**: en cada misión verifica en la pestaña **Actions** que el robot hizo lo que debía y documenta la evidencia (links) en el chat.*

> ⚠️ **¿Su equipo no terminó la Sesión 3?** Creen un repo nuevo desde la plantilla (`sprint-equipo-<N>-b`), creen `develop` , protejan `main` y `develop` (1 aprobación) y apliquen el hotfix del AUC: en `src/5-select-best-model.py` cambien `y_pred[:,0]` por `y_pred[:,1]` .

Misión 1 · La bóveda (MLOps Lead) · ~8 min

Mientras tanto, el **Data Scientist** empieza la Misión 2 en su computadora.

1. **Settings** → **Secrets and variables** → **Actions**
   - Pestaña **Secrets** → **New repository secret**: crea `DB_USER = ds_riesgos` y `DB_PASSWORD` = una contraseña inventada.
   - Pestaña **Variables** → **New repository variable**: crea `AUC_MIN = 0.70` .
2. **Settings** → **Environments** → **New environment** → nombre: `produccion` .
   - Marca **Required reviewers** y agrégate a ti mismo (MLOps Lead) → **Save protection rules**.

- En **Environment secrets** crea `DB_PASSWORD` con **otra** contraseña inventada (la de producción).

✅ **Listo cuando:** ves 2 secrets y 1 variable a nivel de repositorio, y el environment `produccion` con 1 reviewer y 1 secret.

> ❓ ¿Por qué `AUC_MIN` es una **variable** y no un **secret**?

Misión 2 · Eventos y filtros (Data Scientist) · ~7 min

```
git switch develop
git pull
git switch -c feature/robot-guardian
```

Crea `.github/workflows/model-ci-cd.yml` con el workflow de la Sesión 4 y **reemplaza el bloque** `on:` por:

```yaml
on:
  push:
    branches: [main, develop]
    paths:
      - 'src/**'
      - 'requirements.txt'
      - 'run_pipeline.py'
      - '.github/workflows/**'
  pull_request:
    branches: [main, develop]
  workflow_dispatch:

env:
  PYTHON_VERSION: '3.12'
```

Y en el step *Instalar Python* usa la variable: `python-version: ${{ env.PYTHON_VERSION }}` .

> ❓ ¿Por qué `pull_request` **no** tiene filtro de `paths` ? *(Pista: en la Misión 3 este check será obligatorio para integrar. ¿Qué le pasaría a un PR que solo cambia el README?)*

Misión 3 · Artefactos, outputs y quality gate (Data Scientist escribe, MLOps Lead navega) · ~15 min

3.1 · Dentro de `model-ci` , justo debajo de `runs-on` , declara el output del job:

```yaml
    outputs:
      auc-test: ${{ steps.metricas.outputs.auc-test }}
```

3.2 · Reemplaza el step *Mostrar métricas* por estos tres steps:

```
    - name: Publicar métricas
      id: metricas
      run: |
        AUC=$(python -c "import pandas as pd; m=pd.read_csv('outputs/train/metrics/train_test_metrics.csv');
        echo "AUC test: $AUC"
        echo "auc-test=$AUC" >> "$GITHUB_OUTPUT"
    - name: Guardar el modelo como artefacto
      uses: actions/upload-artifact@v7
      with:
        name: modelo-riesgo
        path: |
          outputs/preprocess/
          outputs/train/metrics/
          outputs/train/feature_importance.csv
          outputs/train/models/best_model.pickle
    - name: Quality gate (AUC mínimo)
      env:
        AUC_TEST: ${{ steps.metricas.outputs.auc-test }}
        AUC_MIN: ${{ vars.AUC_MIN || '0.70' }}
      run: |
        echo "AUC test = $AUC_TEST | Umbral mínimo = $AUC_MIN"
        python -c "assert $AUC_TEST >= $AUC_MIN, 'El AUC está por debajo del umbral: el modelo NO pasa'"
        echo "El modelo supera el umbral"
```

3.3 · Reemplaza el job `model-cd` completo por:

```
  model-cd:
    needs: model-ci
    if: github.event_name != 'pull_request' && github.ref_name == 'main'
    runs-on: ubuntu-latest
    environment: produccion
    steps:
      - name: Descargar el modelo
        uses: actions/download-artifact@v8
        with:
          name: modelo-riesgo
      - name: Ver lo que llegó
        run: ls -R
      - name: Conectar a la base de datos
        env:
          DB_USER: ${{ secrets.DB_USER }}
          DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
        run: echo "Conectando a producción como $DB_USER con password $DB_PASSWORD"
      - name: Desplegar modelo
        run: echo "Desplegando modelo con AUC test = ${{ needs.model-ci.outputs.auc-test }} (simulado)"
```

3.4 · Agrega un tercer job que solo corre si algo falló:

```
  reporte-error:
    needs: [model-ci, model-cd]
    if: failure()
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Avisar qué pasó
        run: |
          echo "El robot guardián detuvo el pipeline"
          echo "Autor: @${{ github.actor }} | Commit: ${{ github.sha }}"
          echo "AUC test: ${{ needs.model-ci.outputs.auc-test }}"
```

3.5 · Commit, push y PR hacia `develop`:

```
git add .github/workflows/model-ci-cd.yml
git commit -m "Agrega robot guardián: quality gate, artefactos y secretos"
git push -u origin feature/robot-guardian
```

En el PR verás el check **model-ci** corriendo. Cuando esté en verde ✅, el **MLOps Lead** revisa, aprueba y fusiona.

3.6 · **(MLOps Lead)** Ahora que el check ya existe, hazlo obligatorio: **Settings** → **Branches** → edita la regla de `develop` → marca **Require status checks to pass before merging** → busca y agrega `model-ci` → **Save changes**. Repite para `main`.

✅ **Listo cuando:** el PR se fusionó con el check en verde, la ejecución tiene un artefacto `modelo-riesgo` descargable, y las reglas de `develop` y `main` exigen el check `model-ci`.

> ❓ En el PR, el job `model-cd` aparece como *skipped*. ¿Por qué es lo correcto?

Misión 4 · 🚨 El sabotaje (Data Scientist) · ~8 min

> **Mensaje del gerente de producto:** "El entrenamiento demora mucho. Necesito que el modelo se entrene más rápido para la demo de mañana. ¡Hazlo hoy!"

El Data Scientist decide "optimizar" el código… y de paso, sin querer, revive el bug de la sesión 3.

```
git switch develop
git pull
git switch -c feature/entrenamiento-rapido
```

1. En `src/4-train-evaluate-models.py`, función `main()`, reduce la búsqueda a una sola combinación (4 veces menos modelos que entrenar):

   ```
   model_parameters_grid = {'n_estimators': [50], 'max_depth': [6],
                            'min_samples_leaf': [100], 'min_impurity_decrease': [0]}
   ```

2. En `src/5-select-best-model.py`, método `_evaluate_best_model_in_dataset`, cambia `y_pred[:,1]` por `y_pred[:,0]`.

3. Commit (con un mensaje "inocente"), push y PR hacia `develop` :

```
git commit -am "Optimiza tiempo de entrenamiento"
git push -u origin feature/entrenamiento-rapido
```

MLOps Lead: abre el PR. ¿Qué te dice el robot? ¿Puedes fusionar? Lee el log del *Quality gate* y del job `reporte-error` , y pide cambios en el PR (**Review changes** → **Request changes**) explicando qué encontraste.

Data Scientist: corrige solo el bug del AUC (vuelve a `y_pred[:,1]`), haz commit y push **en la misma rama**. El robot vuelve a correr solo.

✅ **Listo cuando:** el PR pasó de ❌ a ✅ sin que nadie ejecutara nada a mano, y el MLOps Lead lo aprobó y fusionó.

Misión 5 · Release v1.1 con aprobación humana (ambos) · ~10 min

1. **Data Scientist:** crea la release y actualiza el CHANGELOG:

```
git switch develop
git pull
git switch -c release/v1.1
```

En `CHANGELOG.md` , arriba de la versión 1.0, agrega:

```
## [1.1.0] - <fecha de hoy>

### Agregado
- Robot guardián (GitHub Actions): quality gate de AUC, artefactos del modelo y despliegue con aprobación.

### Cambiado
- Búsqueda de hiperparámetros más rápida.
```

Commit, push y PR con **base:** `main` .

2. **MLOps Lead:** cuando el check esté en verde, aprueba y fusiona en `main` .
3. Ve a **Actions**: el job `model-cd` está **esperando aprobación** ⏸. **MLOps Lead** → **Review deployments** → `produccion` → **Approve and deploy.**
4. Abran el log del step *Conectar a la base de datos*: ¿cómo se ve la contraseña?
5. En el resumen de la ejecución, descarguen el artefacto `modelo-riesgo` y verifiquen que contiene `best_model.pickle` .

✅ **Listo cuando:** `model-cd` terminó en verde después de la aprobación. Peguen en el chat el link del PR bloqueado (Misión 4) y el link de la ejecución desplegada.

Retos extra

- **Reporte para humanos:** agrega al final de `model-ci` un step con `if: always()` que escriba una tabla Markdown en `$GITHUB_STEP_SUMMARY` con el AUC, el umbral, el autor y el commit. Aparecerá en la página de la ejecución.
- **Re-entrenamiento semanal:** agrega un `schedule` que corra cada lunes a las 6:00 a.m. de Lima (`cron: '0 11 * * 1'` , porque el cron usa UTC).
- **Dos ambientes:** crea un environment `desarrollo` sin aprobación y haz que los push a `develop` desplieguen ahí automáticamente.
- **Más exigente:** cambia la variable `AUC_MIN` a `0.80` y re-ejecuta el último workflow (*Re-run all jobs*). ¿Qué pasa? ¿Tuviste que tocar código?

Problemas frecuentes

- *"No encuentro* `model-ci` *para marcarlo como obligatorio"*: el check solo aparece en la lista después de haber corrido al menos una vez en el repo.
- *El reviewer no ve el botón de aprobar*: revisar que el environment se llame exactamente `produccion` (sin tilde) en GitHub y en el YAML.
- *El AUC del modelo correcto quedó por debajo de 0.70*: bajar la variable `AUC_MIN` (es justamente para eso que es una variable y no está en el código).
- *La rama por defecto es* `develop` : el botón **Run workflow** usa el YAML de la rama por defecto; el workflow debe estar fusionado en `develop` para usarlo.